

TP6 — La Récursivité

Informations générales

- Filière : PASS Informatique — ESIEA
- Séance concernée : Séance 5 — Récursivité
- Notions travaillées : cas de base, cas récursif, pile d'appels, méthodes récursives, complexité d'un algorithme récursif
- Pré-requis : méthodes, tableaux, notion de complexité (TP1 à TP5)
- Durée indicative : 1h30

Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Expliquer ce qu'est une méthode récursive et identifier son cas de base et son cas récursif
- Écrire des méthodes récursives simples (factorielle, puissance, somme d'un tableau)
- Dérouler "à la main" la pile d'appels d'une méthode récursive
- Reconnaître les situations où la récursivité est coûteuse en complexité
- Comprendre le principe diviser pour régner à travers une implémentation récursive de Quicksort

Exercice 1 — Cas de base et cas récursif

Énoncé

On vous donne la méthode récursive suivante :

```
static void compteAREbours(int n) {  
    if (n == 0) {  
        System.out.println("Decollage !");  
    } else {  
        System.out.println(n);  
        compteAREbours(n - 1);  
    }  
}
```

- a) Quel est le cas de base de cette méthode ? Quel est le cas récursif ?
- b) Qu'affiche l'appel `compteAREbours(3)` ? Écrivez la sortie ligne par ligne.
- c) Que se passerait-il si la condition `if (n == 0)` était supprimée du code ?

Exercice 2 — La factorielle récursive

Énoncé



La factorielle de n (notée $n!$) est définie par : $0! = 1$, et pour $n > 0$, $n! = n \times (n-1)!$. Écrivez une méthode récursive `factorielle(int n)` qui calcule et renvoie $n!$. Testez-la avec $n = 9$.

Exercice 3 — Dérouler la pile d'appels

Énoncé

En reprenant la méthode factorielle de l'exercice 2, déroulez "à la main" l'appel `factorielle(6)` : indiquez chaque appel empilé, puis chaque valeur de retour dépilée, dans l'ordre.

Exercice 4 — Puissance et somme d'un tableau

Énoncé

- a) Écrivez une méthode récursive `puissance(int base, int exposant)` qui calcule $\text{base}^{\text{exposant}}$, sachant que $\text{base}^0 = 1$ et $\text{base}^n = \text{base} \times \text{base}^{(n-1)}$ pour $n > 0$. Testez avec `puissance(2, 5)`.
- b) Écrivez une méthode récursive `sommeTableau(int[] tab, int indice)` qui calcule la somme des éléments du tableau à partir de `indice` jusqu'à la fin. Le cas de base est `indice == tab.length` (somme vaut alors 0). Testez avec le tableau `{4, 8, 2, 6}` en appelant `sommeTableau(tab, 0)`.

Corrigé

a) Puissance récursive :

```
static int puissance(int base, int exposant) {
    if (exposant == 0) {
        return 1;
    } else {
        return base * puissance(base, exposant - 1);
    }
}
```

// puissance(2, 5) renvoie 32

b) Somme récursive d'un tableau :

```
static int sommeTableau(int[] tab, int indice) {
    if (indice == tab.length) {
        return 0;
    } else {
        return tab[indice] + sommeTableau(tab, indice + 1);
    }
}
```

// sommeTableau({4, 8, 2, 6}, 0) renvoie 20

Point à retenir : dans les deux cas, le paramètre qui évolue à chaque appel (exposant, puis indice) se rapproche du cas de base — c'est ce qui garantit que la récursion se termine.

Exercice 5 — Fibonacci récursif et ses limites

Énoncé

La suite de Fibonacci est définie par $\text{fib}(0) = 0$, $\text{fib}(1) = 1$, et $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ pour $n > 1$. Écrivez une méthode récursive `fib(int n)` qui calcule le n -ième terme de la suite, puis affichez `fib(10)`.

Corrigé

```
static int fib(int n) {  
    if (n == 0 || n == 1) {  
        return n;  
    } else {  
        return fib(n - 1) + fib(n - 2);  
    }  
}
```

// fib(10) renvoie 55

Ici, chaque appel à `fib(n)` déclenche deux nouveaux appels récursifs (`fib(n-1)` et `fib(n-2)`), qui eux-mêmes en déclenchent chacun deux autres, et ainsi de suite : le nombre total d'appels croît de façon exponentielle avec n , en $O(2^n)$. Essayez `fib(40)` : le programme prend plusieurs secondes, alors qu'il ne calcule qu'un seul nombre !

Point à retenir : la récursivité rend parfois le code élégant, mais pas forcément efficace. Comme vu en Séance 4 sur la complexité, un algorithme peut être correct tout en étant beaucoup trop lent en pratique — ici à cause d'un très grand nombre de calculs répétés inutilement (`fib(3)` par exemple est recalculé de nombreuses fois).